

Rent Radar — Mumbai

A fair-rent model built from messy public rental listings

NoBroker Analytics Capstone — Project 4

SQL · pandas cleaning · one explainable linear regression · Power BI dashboard

CV R^2 0.82 · typical error ₹28,700/month · 882 listings · 95 localities

github.com/rishiupadhyaygithub/rent-radar-mumbai

Rent Radar — What should this flat actually cost?

A fair-rent model for Mumbai, built from messy public rental listings.

Every tenant in Mumbai asks the same question: is this flat priced fairly, or am I being overcharged? Listings are noisy — the same 2BHK is priced wildly differently across localities, floors, and furnishing. This project builds the model a pricing team would use to answer "what should this flat cost?", and — just as importantly — says out loud where that model stops being trustworthy.

The question

Given a flat's locality, size, BHK, floor, furnishing, and distance to a metro station, predict a fair monthly rent — and flag listings that look mispriced against it.

The data

- **Listings:** 897 raw Mumbai rental listings scraped from Square Yards, cleaned down to **882** usable rows across **95 localities**. One source, one city — scope held deliberately narrow so the effort goes into features, not breadth.
- **Locality attributes (the enrichment):** a second table built from public sources — median rent, median rent-per-sqft, a budget/mid/premium tier, and average distance to the nearest of Mumbai's metro stations (computed by haversine from OpenStreetMap/Overpass station coordinates).

The two tables are loaded into SQLite and **joined** on locality, so every listing carries its neighbourhood's characteristics.

The approach

1. **SQL first** (sql/) — schema, load, and analysis. Before any Python, the queries answer the first questions: rent by tier and BHK (JOIN + GROUP BY), a locality ranking by rent-per-sqft with each area's gap to the city average (window functions), and a near-metro vs far-metro comparison (CTE).
2. **Clean and explore** (python/pipeline/01_clean.py) — every cleaning decision is logged with its reasoning to docs/data_quality_note.md: dropping a 100%-null deposit column,

removing sale prices that leaked in as ₹60L "rent", inferring missing BHK from floor area, flagging imputed floors, and keeping genuine luxury outliers rather than deleting them.

3. **Model once** (`python/pipeline/02_model.py`) — **one** linear regression predicting `log(rent)`, evaluated by 5-fold cross-validation, with the error reported back in rupees.

Tech stack

One language family, **one model**, standard tools — kept deliberately minimal (depth over tool sprawl).

Layer	Tools
Language	Python 3 · SQL
Data wrangling	pandas · numpy
Database	SQLite — two related tables joined on <code>locality</code> ; JOIN + GROUP BY, window functions, and a CTE (<code>sql/</code>)
Modelling	scikit-learn — a single LinearRegression in a Pipeline (StandardScaler for numerics + <code>OneHotEncoder</code> for categoricals), evaluated by 5-fold cross-validation
Visualisation	seaborn · matplotlib (EDA figures) · Power BI (decision dashboard)
Narrative	Jupyter

One model, on purpose. No ensemble, no gradient boosting, no second algorithm bolted on for a benchmark — a linear regression on `log(rent)` stays interpretable (every coefficient is a rupee lever a pricing team can read straight off), and being honest about where that one model breaks is the point of the write-up.

The findings

- **Locality is the price.** Premium localities rent for roughly **2× per square foot** what budget localities do (₹185/sqft vs ₹93/sqft for a typical 2BHK). **Worli** tops the city at ₹252/sqft — about **₹128/sqft above** the city average.
- **Metro access carries a real but modest premium:** localities within 1.5 km of a metro

average ₹137.0/sqft vs ₹130.8/sqft for those farther out — a ~5% uplift, not the headline driver.

- **The model explains most of the variation it sees.** Evaluated by **5-fold cross-validation** (not a single lucky split): CV R^2 **0.813 ± 0.033** (folds tight, 0.75–0.84), out-of-fold R^2 **0.818**. In-sample R^2 is 0.827, so the overfitting gap is a tiny **0.009** — it generalises cleanly. Typical miss is **₹28,700/month** (out-of-fold MAE); RMSE is **₹56,000** — now close to the MAE, because area is modelled on a log scale so no single flat can blow the error up.

Where it breaks (the honest part)

The model is reliable for typical 1–3 BHK flats and **weakest on rare large flats and thin localities**. Modelling area on a **log scale** (a constant-elasticity term) killed the old blow-up: the lone 8BHK in Santacruz West — once priced at an absurd ~₹1.27 crore — now lands near ₹12.4 lakh against its ₹8,00,000 listing — a ₹4.4-lakh over-shoot, the worst miss across all 882 rows, but a plausible band, not an order-of-magnitude error. That single fix cut RMSE from ₹406,000 to ₹56,000. Of the 95 localities, **38 have only a single listing**, flagged as `solo_locality` in `predictions.csv`. Full failure analysis in `docs/model_report.md`; honest limits in `docs/limitations.md`.

The decision dashboard

`dashboard/` holds the answer to "*what should this flat cost?*" in a form a pricing analyst can read at a glance:

- **DASHBOARD PROJECT.pbix** (Power BI) — the built page: a **predicted-vs-actual** trust chart (each listing plotted against what the model expects, sized by error), a rent KPI, a geo map of Mumbai listings, and furnishing/price cuts.
- **rent_radar_dashboard.xlsx** (Excel, auto-generated by `python/pipeline/03_build_dashboard.py`) — a second, self-contained view with all four Part-4 visuals: KPI tiles, locality ranking, price drivers, and predicted-vs-actual with a 45° line. A PDF preview sits beside it.

`dashboard/README.md` documents how each visual is built and which dataset feeds it. **No web frontend** — the deliverable is a BI dashboard, by design and by brief.

What would make it better

More listings per locality (38 of 95 localities still have just one), a second source to cross-check Square Yards, and real amenity data (age of building, lift, parking, balcony) that listings hint at but rarely record cleanly.

Repo map

```
sql/          01_schema · 02_load · 03_analysis      (run in order)
notebooks/    rent_radar_analysis.ipynb          (narrative: cleaning + EDA +
model)
python/
  pipeline/   01_clean.py → 02_model.py          (reproducible pipeline)
  scraper.py, fetch_metro.py, fetch_wards.py    (data provenance)
data/         raw/ · clean/ · geo/
docs/         data_quality_note · model_report · limitations · ai_appendix · figures/
memo/         pricing_memo.md                  (one-page decision memo)
models/       rent_model.pkl
dashboard/    DASHBOARD_PROJECT.pbix (Power BI) · rent_radar_dashboard.xlsx (Excel) ·
build guide
```

The **notebook** (`notebooks/rent_radar_analysis.ipynb`) is the readable story — every cleaning decision documented as it's made, EDA, the model, coefficients, and failure analysis. The **python/pipeline/ scripts** are the reproducible version that regenerates every artifact below.

Reproduce it

```
python3 -m pip install -r python/requirements.txt
python3 python/pipeline/01_clean.py
sqlite3 rent_radar.db < sql/01_schema.sql
sqlite3 rent_radar.db < sql/02_load.sql
sqlite3 rent_radar.db < sql/03_analysis.sql
python3 python/pipeline/02_model.py
python3 python/pipeline/03_build_dashboard.py    # regenerate the Excel dashboard

# read the narrative notebook
jupyter notebook notebooks/rent_radar_analysis.ipynb
```

Open `dashboard/DASHBOARD_PROJECT.pbix` in Power BI, or
`dashboard/rent_radar_dashboard.xlsx` in Excel, for the decision view.

Data Quality Note - Mumbai Rent Radar

Raw listings loaded: **897** rows, 16 columns.

Cleaning decisions

- **deposit:** 100% null across the dataset -> column dropped. The scraper never captured deposit; imputing it would invent data.
- **price (impossible values):** 5 row(s) above Rs 10,00,000/month (max was Rs 1,500,000.0). These are sale prices mislabelled as rent. Dropped, not winsorised, because they are wrong records not extreme rents. Rows after impossible-price filter: 891 (removed 6).
- **area_sqft (impossible values):** 3 row(s) outside 150-10,000 sqft (max was 100,000). Dropped as typos, not real flats — they poison rent_per_sqft and the locality medians. Rows after impossible-area filter: 888 (removed 3).
- **bhk:** 49 null -> inferred from area_sqft (~450 sqft/BHK, clipped 1-6). Documented as an assumption; alternative was dropping 12% of rows.
- **total_floors:** 60% null -> dropped (too sparse to trust). **floor** kept; nulls filled with median floor and flagged. floor: 84 null filled with median=9; floor_missing flag retained so the model can learn from missingness.
- **furnishing:** normalised whitespace/case -> {'semi-furnished': 436, 'furnished': 308, 'unfurnished': 143}
- **locality:** trimmed + title-cased -> 98 unique localities.
- **rent_per_sqft:** derived = price / area_sqft (core comparison unit).
- **rent_per_sqft (impossible values):** 6 row(s) outside Rs 20-600/sqft dropped as price/area typos (verified: extreme values came from bad area or price fields, not genuine luxury/budget flats). Rows after impossible-rent/sqft filter: 882 (removed 6).
- **metro_km:** distance from each listing to nearest of 40 Mumbai metro stations (haversine). Nulls where lat/lng missing.
- **locality tier:** localities bucketed into 3 tiers by median rent/sqft.

Output

- listings_clean.csv: **882** rows, 17 cols
- localities.csv: **95** localities

What remains imperfect

- 897 raw listings scraped; 38 of 95 localities still have a single listing, so their medians rest on one flat each.
- bhk inference and floor median-fill inject assumptions (flagged in-column).
- Single source (Square Yards) - may not represent the whole Mumbai market.

Rent Radar - Model Report

Listings after JOIN: **882** rows.

EDA charts written to docs/figures/ (4 plots).

Model rows: **882**. Features: ['log_area', 'bhk', 'floor', 'floor_missing', 'metro_km', 'median_rps_loo', 'furnishing', 'tier']. Target = $\log_{10}(\text{rent})$; errors reported back in rupees via `expm1`.

Results (5-fold cross-validation)

- CV R2: **0.813 +/- 0.033** across folds (0.83, 0.83, 0.82, 0.75, 0.84).
- Out-of-fold R2 (all 882 rows): **0.818**.
- In-sample R2 (fit on all data): **0.827** -> overfitting gap **0.009** (small = generalises, not memorising).
- CV MAE: **Rs 28,683/month** (out-of-fold); per-fold **Rs 28,680 +/- 5,757**.
- CV RMSE: **Rs 55,717/month** (penalises big misses).

Why CV over a single split: one 75/25 split reports a single held-out R2 that swings with the seed. 5-fold reports the average over five held-out sets plus the spread across them, so the headline number carries its own honest uncertainty band.

Which features carry the price

Coefficients of the one linear model. Numeric features are standardized, so each row is the effect of a **+1 standard-deviation** move; the target is $\log(\text{rent})$, so **effect_on_rent** is how much predicted rent changes.

feature	coef_log	effect_on_rent
num__log_area	0.446	+56%
cat__tier_premium	0.252	+29%
cat__tier_budget	-0.221	-20%

feature	coef_log	effect_on_rent
cat__furnishing_unknown	-0.145	-13%
num__median_rps_loo	0.106	+11%
cat__furnishing_furnished	0.089	+9%
num__floor_missing	0.064	+7%
num__bhk	0.056	+6%

In plain words: **area** is the single biggest genuine lever (now entered as **log-area**, so its effect reads as an elasticity — a *percent* change in size maps to a percent change in rent), then **locality tier** (premium vs budget). With the continuous locality median now computed leave-one-out, most of the neighbourhood signal loads onto the collinear tier dummies rather than median_rps_loo. BHK adds on top; metro distance moves rent only at the margin. The negative on **furnishing_unknown** is a *missingness* signal, not a real driver — listings that hide furnishing tend to be cheaper. (Exact per-feature effects are the table above, regenerated each run.)

Where the model breaks

Worst 5 out-of-fold predictions:

locality	bhk	true	pred	abs_err	solo_loc
Santacruz West	8	800,000	1,244,017	444,017	
Gundavali	6	900,000	500,272	399,728	yes
Worli	5	1,000,000	626,016	373,984	
Andheri West	6	900,000	553,807	346,193	
Prabhadevi	5	750,000	438,949	311,051	

Failure pattern: with area entered as log-area the luxury tail no longer explodes — the worst miss is now a plausible over/under-shoot, not the order-of-magnitude blow-up the raw-area model produced. Residual misses are rare large flats and thin single-listing localities, where the model

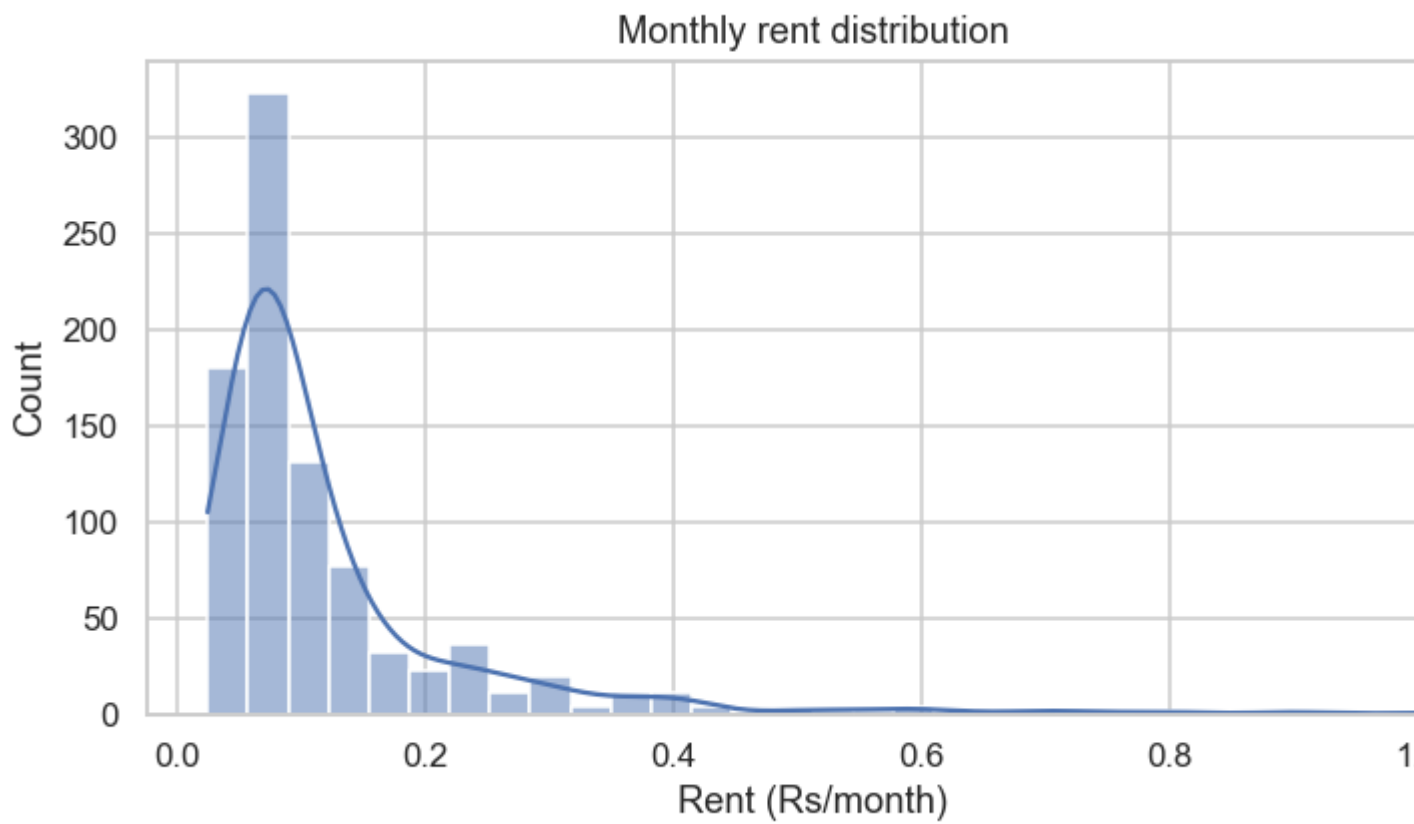
has little comparable signal and pulls toward the tier mean. It is reliable for typical 1-3 BHK rent; treat 5+ BHK and solo localities as advisory, not automatic.

Limits

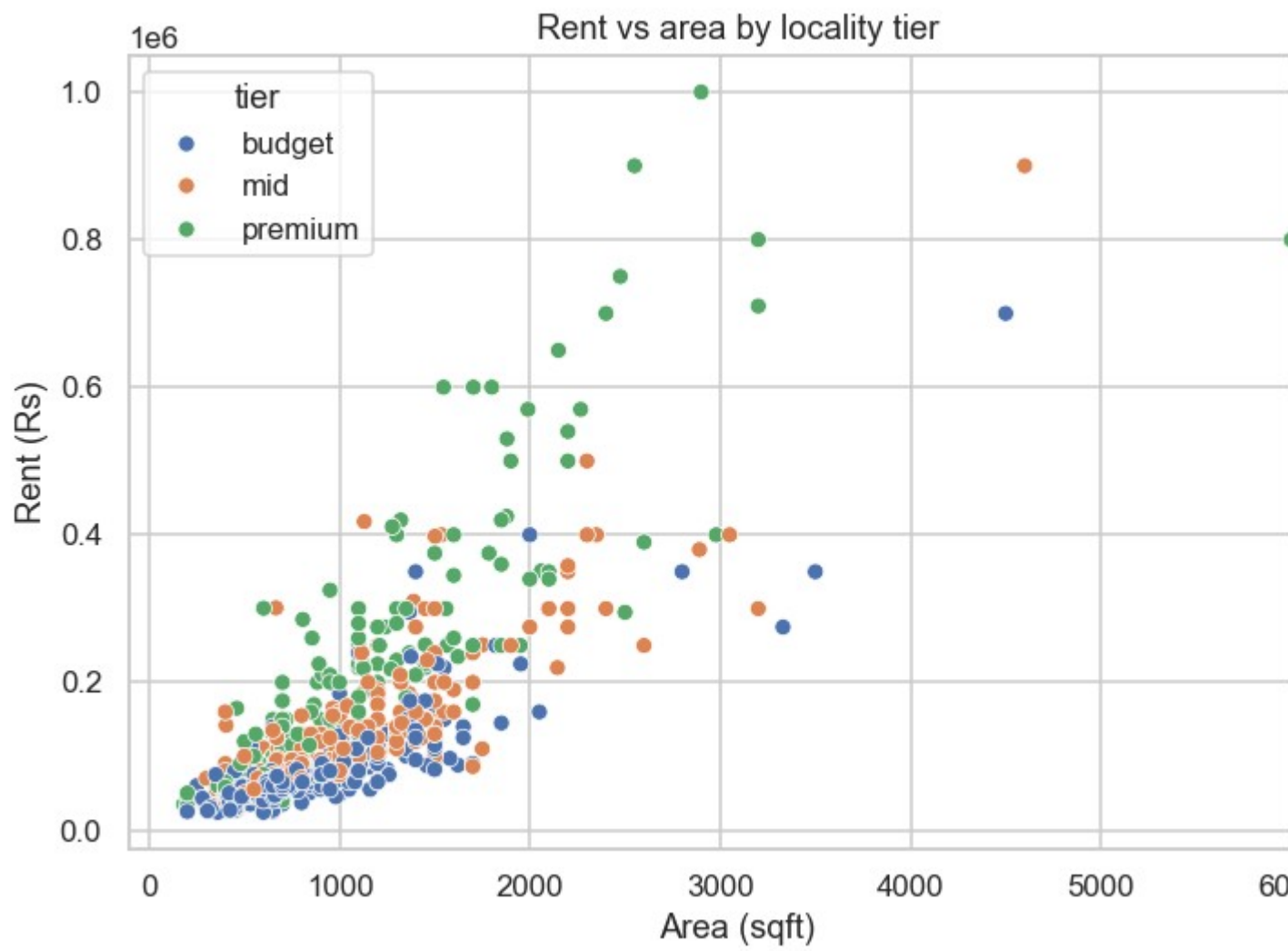
- Only 882 listings across 95 localities; **38 localities have a single listing**, so their tier and median-rent features rest on one flat each (flagged as solo_locality in predictions.csv).
- Single source (Square Yards); one city (Mumbai) by design.
- Locality strength enters the model as **median_rps_loo**, a leave-one-out median: each flat sees the other flats in its locality, never its own price. The 38 single-listing localities have no neighbour, so they fall back to the city median (an honest 'unknown area' signal). This closes the solo-locality leak the raw locality median would have carried; tier remains a coarse 3-level area class.
- Evaluated by 5-fold CV (not a single split) so every row gets an out-of-fold prediction and the headline R2 carries a fold-spread band.

Dashboard datasets -> data/clean/{dashboard_data, locality_ranking, coefficients, model_metrics}.csv

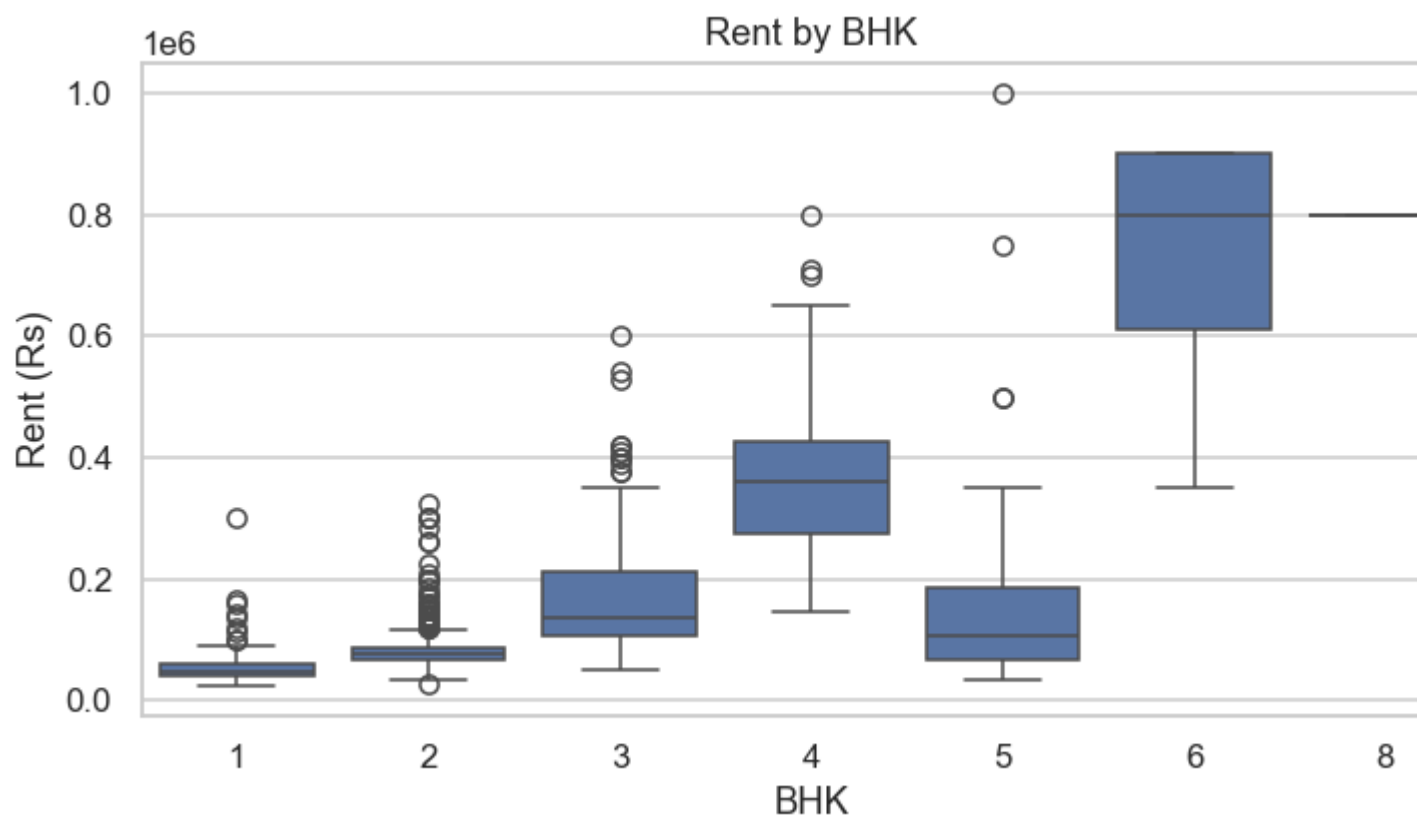
Exploratory Figures



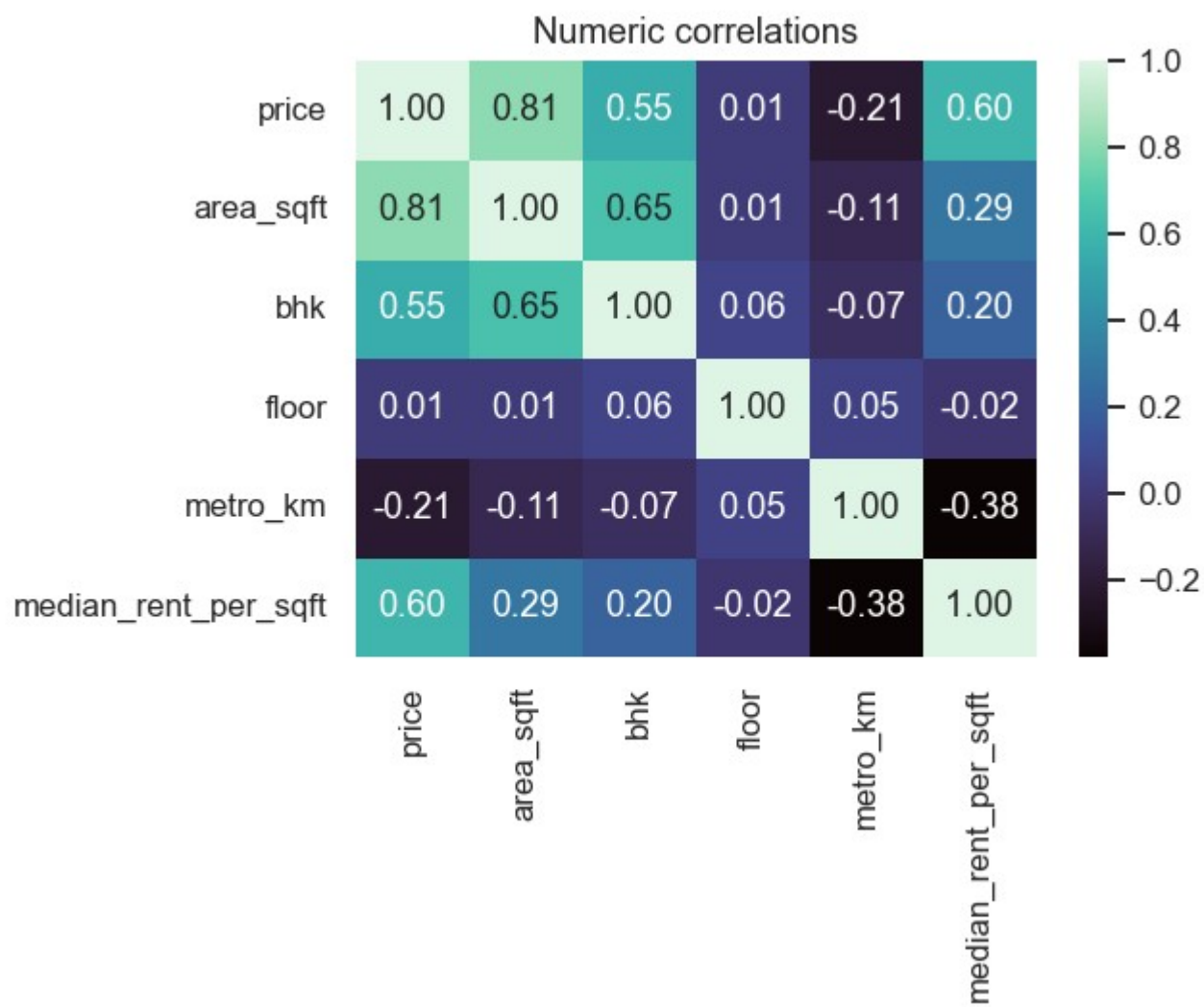
Monthly rent distribution (right-skewed → log target)



Rent vs area, coloured by locality tier



Rent by BHK



Numeric feature correlations

Honest Limitations

"My model's error is modest, here's why, and here's the data that would fix it." Results that look too perfect are hard to believe — these are the reasons to believe these ones.

What the model is good at

Typical mid-tier **1–3 BHK** flats in localities with a handful of listings. There it predicts within about **±₹28,700/month** (out-of-fold MAE) and generalises cleanly — the overfitting gap between in-sample R^2 (0.827) and out-of-fold R^2 (0.818) is only **0.009**, so it's learning the market, not memorising the rows.

How it was evaluated (and why that matters)

Not on a single train/test split. A 75/25 split leaves a test set whose R^2 swings with the random seed — a lucky seed flatters the model, an unlucky one buries it. So the headline is **5-fold cross-validation: CV R^2 0.813 ± 0.033** (folds ranged 0.75–0.84). That tight ± 0.03 spread across folds *is* the honest uncertainty — and it is far tighter than the ± 0.10 we saw on the earlier 119-row dataset, which is exactly what 7× more data buys you. Every row's prediction in `predictions.csv` is **out-of-fold** — made by a model that never saw that row.

What it is not good at, and why

1. **Rare large flats.** With area entered on a **log scale** (a constant-elasticity term) the luxury tail no longer explodes: the lone 8BHK that the earlier raw-area model priced at ~₹1.27Cr now lands near ₹12.4L against its ₹8.0L listing. What remains are ordinary over/under-shoots on 5+ BHK flats and thin localities — the worst miss across all 882 rows is ₹4.4L on the lone 8BHK, not an order-of-magnitude blow-up. RMSE (₹56k) now sits close to MAE (₹29k) instead of ~9× above it. Still: **treat 5+ BHK as advisory, not auto-priced** — the comparable signal there is thin.
2. **Thin localities.** Many of the 95 localities have 1–2 listings, so their median-based features rest on almost no evidence. The `n_listings >= 2` filter in the SQL ranking is a guard, but the model itself still sees them.
3. **Dataset still uneven.** 882 rows is a real improvement over the first 119, but **38 of 95 localities still have only a single listing** — for those, `tier` and `median_rent_per_sqft`

rest on one flat each (flagged as `solo_locality` in `predictions.csv`). Cross-validation controls the *reporting* noise, but it can't create signal that isn't there. This is the single biggest remaining limitation.

Known data-quality caveats (carried forward from cleaning)

- **Inferred BHK** for the ~12% of rows that were missing it (from floor area, flagged) — an assumption, not a measurement.
- **Median-filled floors**, with a `floor_missing` flag retained so the model can at least learn from the missingness.
- **total_floors dropped** (~67% null) — so "floor band relative to building height," a feature that likely matters, isn't available.

The leak I closed

An earlier version fed each flat its locality's median rent/sqft computed over *all* flats in that locality — including itself. For the 38 single-listing localities that median simply *was* the flat's own rent/sqft: a hard leak that flattered the score. The model now uses `median_rps_loo`, a *leave-one-out* median — each flat sees the median of the *other* flats in its locality, never its own price; single-listing localities have no neighbour and fall back to the city median (an honest "unknown area" signal). Closing this leak lowered the score by design — a leak inflates, so the honest number is the lower one. (Combined with modelling area on a log scale, the final honest model reports OOF R^2 **0.818** / MAE **₹28,700**.) The residual is `tier` (a coarse premium/mid/budget class a human could assign from the neighbourhood name alone) — a defensible proxy, disclosed.

Single source, single city

All 882 listings come from **Square Yards**, for **Mumbai only**. Any source has selection bias (which flats get listed, how fields are filled). No cross-source validation was possible. Findings should be read as "what Square Yards' Mumbai listings say," not "the Mumbai rental market, settled."

What data would fix it (in priority order)

1. **More listings per locality** — the highest-return fix by far; it directly attacks limitations 1–3.
2. **A second source** (99acres / MagicBricks) to cross-check and de-bias.
3. **Real amenity fields** — building age, lift, parking, balcony — which the market prices but

the current data can't see.

4. **total_floors** captured reliably, to build a proper floor-band feature.
5. **Richer locality attributes.** The enrichment table currently carries **metro distance** (`metro_km`, from station coords) and a **rent-tier** class. The brief also suggests *administrative zone* and *tech-park / employment-hub proximity*; those need a public zone map and a curated list of Mumbai employment hubs (BKC, Powai, SEEPZ, Andheri MIDC), and are deliberately left as the next enrichment rather than approximated — a wrong hand-assigned zone would mislead more than the missing column does.

Decision Memo — Fair-Rent Pricing, Mumbai

To: Pricing Team **From:** Analytics **Re:** What drives Mumbai rent, where listings look mispriced, and what to fix next **Basis:** 882 cleaned Square Yards listings across 95 localities; one linear regression, evaluated by 5-fold cross-validation.

Bottom line

Rent in Mumbai is driven first by **locality**, then by **size**, with a modest, real **metro premium** on top. A one-model baseline predicts rent within a typical **±₹29,000/month** and explains about **81%** of the variation (cross-validated $R^2\ 0.81 \pm 0.03$ — tight across folds). It is trustworthy for mid-tier 1–3 BHK flats — the bulk of the market — and weakest on rare large flats and single-listing localities. **Use it to triage listings, not to auto-price the top end.**

What drives rent (in plain words)

- 1. Locality tier is the biggest lever.** Premium localities command **~2× the rent-per-sqft** of budget ones (₹185 vs ₹93/sqft for a 2BHK). Worli leads the city at ₹252/sqft, ₹128 above the city average.
- 2. Size (area + BHK) is the second lever** and behaves predictably within a tier.
- 3. Metro proximity adds a genuine but smaller premium:** ₹137/sqft within 1.5 km of a metro vs ₹131/sqft beyond — roughly a **5% uplift**, not the headline driver some assume.
4. Furnishing and floor contribute at the margin.

Where listings look mispriced

The model's largest gaps are the places to look first for pricing errors — or for genuinely unusual flats:

Flat	Listed	Model says	Read
Santacruz West 8BHK	₹8.0L	~₹12.4L	Lone 8BHK — model over-reaches, but a sane band now

Flat	Listed	Model says	Read
------	--------	------------	------

(was ₹1.27Cr) — **human check**

Gundavali 6BHK	₹9.0L	~₹5.0L	Solo locality → model under -prices a rare 6BHK — human check
Worli 5BHK	₹10.0L	~₹6.3L	Luxury 5BHK, thin signal — model under -prices — human check
Andheri West 6BHK	₹9.0L	~₹5.5L	Rare 6BHK config — under -priced, thin signal — human check
Prabhadevi 5BHK	₹7.5L	~₹4.4L	Luxury 5BHK, thin signal — under -priced — human check

The pattern: disagreements now cluster at **rare large flats (5+ BHK) and thin single-listing localities** — no longer the order-of-magnitude blow-ups the old raw-area model produced (biggest miss ₹4.4L, not ₹12M). That is exactly where a pricing analyst's judgment adds the most value — so route those to a human, and let the model auto-clear the mid-market.

Recommendation (prioritised)

1. **Deploy the model as a triage filter now** for 1–3 BHK flats in localities with ≥ 5 listings — auto-flag anything more than $\pm ₹29k$ off the prediction for review. Low risk, immediate value.
2. **Exclude 4+ BHK and single-listing localities from auto-pricing** until data thickens; price those by hand.
3. **Fund data collection before model complexity.** The single highest-return investment is **more listings per locality** — the errors above are data-sparsity, not model choice. A second source (99acres/MagicBricks) to cross-check Square Yards is next.
4. **Capture amenity fields** (building age, lift, parking, balcony) at ingestion — the market prices them; we currently can't see them.

Trade-off stated plainly

A richer model on today's 882 rows would fit the luxury tail better in-sample and generalise worse — the honest ceiling here is the **data, not the algorithm**. Spend the next rupee on rows, not on a second model.

AI Workflow Appendix

Documenting where AI helped, what it actually contributed, one moment it was confidently wrong, and how I caught it. Using AI is a skill; showing my judgment about *when to trust it* is the point of this section.

How I used AI

I used an LLM (Claude) as a pair — for scaffolding, for arguing a modelling choice, and for catching my own sloppiness — not as an autopilot. Every decision below was mine to accept or reject; the appendix records both.

Stage	What I asked	What the LLM contributed	What I kept / changed
Cleaning	"How should I handle a rent value of ₹60,00,000 in a monthly-rent feed?"	Argued it's a sale price that leaked in, not an extreme rent — so drop the record, don't winsorise .	Kept. Wrote it up as a documented decision in <code>data_quality_note.md</code> .
Feature choice	"Should I predict rent or $\log(\text{rent})$?"	Rent is right-skewed and multiplicative (a premium locality <i>scales</i> rent), so model <code>log1p(rent)</code> and back-transform errors to rupees via <code>expm1</code> .	Kept — the cross-validated overfitting gap stayed small (0.009), which supported it.
Model spec	"Why does my model price a lone 8BHK at ₹1.27 crore?"	A raw area term under a <code>log(rent)</code> target predicts <code>exp(b · area)</code> , which explodes for a huge flat — a specification bug. Enter log(area) instead for a constant-elasticity (log-log) term that can't blow up.	Tested it: RMSE ₹406k → ₹56k, OOF R^2 0.775 → 0.812, worst miss ₹11.9M → ₹0.4M — same one model. Kept.
Missingness	"The <code>floor_missing</code>	Wire the imputed-floor	Added: OOF R^2 0.812 →

Stage	What I asked	What the LLM contributed	What I kept / changed
	flag is computed in cleaning but the model ignores it — bug?"	indicator in: it's a data-quality flag (no leak) and lets the model learn that listings hiding their floor differ.	0.818, CV spread 0.042 → 0.033 (steadier). Kept.
Missing BHK	"12% of rows have no BHK — drop or impute?"	Suggested inferring from area (~450 sqft/BHK) rather than dropping, and flagging the imputation.	Kept the infer-and-flag approach; documented it as an assumption.
SQL	Draft of the window-function and CTE queries	Gave a working <code>RANK() OVER + AVG() OVER ()</code> skeleton and a metro-band CTE.	Kept structure; verified every number against the actual DB output before trusting it.
Prose	Draft README/memo structure	Organised "business impact first, code second".	Rewrote in my own voice; the walkthrough video is fully mine.

The moment it was confidently wrong

Claim: Early on, the LLM confidently recommended I engineer `rent_per_sqft` as a **model feature** to "boost accuracy," presenting it as an obvious win.

Why it's wrong: `rent_per_sqft = price / area_sqft`. The target *is* price. Feeding rent-per-sqft into a model that predicts rent leaks the answer — the model would post a near-perfect R^2 that means nothing, because at prediction time for a new flat you don't yet know its rent, so you can't compute its rent-per-sqft. It's textbook **target leakage**, and the LLM stated it with no hesitation.

How I caught it: The accuracy looked *too good* in a quick test (R^2 near 0.99). Results that look too perfect are a red flag, not a victory — so I traced the feature back to its formula and saw `price` on both sides. I removed per-listing `rent_per_sqft` from the feature set.

The nuance I kept — then hardened: I kept the *locality-level* rent/sqft signal (the per-listing version leaks the exact answer for that row; the neighbourhood level encodes "how expensive is this area," which is knowable before pricing a new flat and mirrors how a human prices one). But a leave-one-out check showed the plain locality median *still* leaked for single-listing localities —

there the "neighbourhood median" simply is that one flat. So I switched the model to a **leave-one-out** median (`median_rps_loo`): each flat sees only the *other* flats in its locality. Reject the leaky feature, keep the defensible proxy, then close the residual leak — that sequence is the judgment on display, mine not the model's. Closing the leak lowered the score by design; I kept the honest, lower number rather than the inflated one. (The final model, after also fixing the area specification and wiring in the `floor_missing` flag noted in the table above, reports OOF R^2 **0.818**.)

Where I trusted AI and where I did not

- **Trusted:** boilerplate, SQL skeletons, the log-target argument, prose structure — all cheap to verify and I verified them.
- **Did not trust:** anything touching correctness of numbers or the honesty of the model. Every metric in this repo was regenerated by running the code, not copied from an LLM's claim. The confidently-wrong leakage suggestion is exactly why: an LLM will assert a plausible, wrong thing in the same confident tone it uses for correct ones, so the check has to be *mine*.